

Bloodrose — Operator Surface Spec

Governed Observation & Control Layer — Version 0.1

Classification: Canonical Governance Document

Audience: System Architects · Pipeline Operators · QA Leads · Governance Auditors

Author: System Architect

Date: April 21, 2026

Status: Pending Review

1. Purpose Statement

The Operator Surface is the governed observation and control layer for the Bloodrose system. It is not a player-facing interface. It is the cockpit through which the System Architect, project lead, or designated operator observes, audits, and controls the running Bloodrose system in real time. Where the player sees atmosphere, narrative, and expressive interaction, the Operator Surface exposes the raw machinery beneath — state variables, event processing pipelines, canonical compliance status, and system health indicators. Every readout on this surface exists to answer a single question: *Is the system behaving as specified?*

The Operator Surface exists because governed systems require operator-level auditability. The player sees the game. The operator sees the truth beneath it. Without this layer, drift goes undetected, violations go unlogged, and canonical integrity becomes a matter of faith rather than evidence. The Operator Surface transforms Bloodrose from a black-box game into an inspectable, auditable, verifiable system — one where every state transition, every event fire, and every threshold crossing is visible, traceable, and accountable to the canonical specifications that define it.

"Atmosphere may be expressive. Logic must remain explicit."

— Bloodrose Governance Principle

In v0.1, the Operator Surface may manifest as a debug overlay, a separate developer panel, or a structured console view. Its form is provisional; its function is non-negotiable. In later phases, it evolves into a full cockpit UI with dedicated panels, real-time compliance dashboards, session replay, and export capabilities. Regardless of phase, the governing contract is the same: the Operator Surface must faithfully reflect the canonical state of the system, must never silently alter that state, and must provide an unbroken audit trail for every session it observes.

2. Design Principles

The following principles govern the design, implementation, and operation of the Operator Surface across all phases. No feature or panel may violate these principles.

1. **Transparency over abstraction** — Every state variable, event fire, and transition must be observable, not hidden behind UI polish. The Operator Surface displays raw values, not interpreted summaries.
2. **Read-first, write-guarded** — The operator can always observe. Write and override actions require explicit confirmation and are permanently logged. Observation is a right; mutation is a governed privilege.
3. **Canonical traceability** — Every displayed value must map to a canonical document — a spec-defined variable, an Event Table entry, a governance rule. If a value cannot be traced, it does not belong on the surface.
4. **No silent state** — If state changes, the Operator Surface must reflect it within the same tick or frame. Delayed, batched, or deferred display of state changes is a violation of this principle.

- 5. **Deterministic replay** — The Operator Surface must support state snapshot capture and deterministic replay for debugging and post-session analysis. Full implementation targeted for Phase 2 and beyond.
 - 6. **Separation of concerns** — The Operator Surface never modifies player-facing rendering. It observes the same state the player UI consumes but displays it in raw, unglamoured form. The operator layer and the player layer are parallel consumers of a single source of truth.
-

3. Surface Architecture Overview

The Operator Surface is composed of five major panels, each serving a distinct observational or analytical function. Together, they provide complete visibility into the running Bloodrose system.

Panel	Function	Primary Audience
State Inspector	Real-time readout of all canonical state variables	System Architect, Pipeline Operators
Event Log	Timestamped, scrolling record of every event fired	All operators
Canonical Compliance	Live compliance dashboard with drift detection	QA Leads, Governance Auditors
Entity Inspector	Detailed view of individual game entities on selection	Pipeline Operators, QA Leads
Session Summary	End-of-session report with full audit trail	All operators, Governance Auditors

3.1 State Inspector Panel

The State Inspector provides a real-time readout of all canonical state variables. Values update on every tick. Each variable is displayed with its current value and, where applicable, color-coded threshold bands for rapid visual assessment.

Variable	Type	Range	Threshold Bands
corruption	int	0-100	0-49 Green · 50-74 Amber · 75-99 Red · 100 Critical/Black
redemption	int	0-100	0-29 Dim · 30-64 Emerging · 65+ Active/Gold
score	int	0-∞	Running total — no threshold bands
player_position	x, y	Canvas bounds	Coordinate readout — no threshold bands
game_over	boolean	true / false	Binary flag — highlighted on true
ending_type	string / null	null, "consumed", "mercy_dominant", "bound_unbroken"	Displayed only when non-null
Petal instances remaining	int (by type)	0-n	Breakdown: Guilt, Desire, Rage, Mercy
Thorn zone count	int	0-n	Total active hazard zones
Altar status	enum	locked / unlocked	Derived from redemption ≥ 65

3.2 Event Log Panel

The Event Log is a scrolling, timestamped, append-only record of every event that fires during the session. It is the definitive audit trail for session activity.

Column Structure:

Column	Description
Timestamp	Game tick or real-time clock value at event fire

Column	Description
Event ID	Canonical EVT-ID from the Event Table
Event Name	Human-readable event name
Category	Event classification (see color codes below)
Trigger Condition Met	The condition expression that evaluated to true
State Effects Applied	List of state mutations resulting from this event
Narrative Output Dispatched	Narrative log text sent to the player layer, if any

Category Color Codes:

Category	Color	Usage
COLLECT	Blue	Petal collection events
HAZARD	Red	Thorn contact, corruption gain events
THRESHOLD	Amber	Corruption or redemption crossing a threshold band
TERMINAL	Crimson Pulse	Game-ending events — full-width alert bar highlight
NARRATIVE	Silver	Narrative log dispatch events

Behavior:

- Filterable by category — operator may show/hide any combination of categories
- Terminal events are highlighted with a full-width alert bar spanning the log panel
- The log is append-only during a session; no entries may be deleted or modified
- Exportable as JSON or CSV for post-session analysis

3.3 Canonical Compliance Panel

The Canonical Compliance Panel is a live dashboard that continuously validates the running system against canonical specifications. It is the governance heartbeat of the Operator Surface.

Compliance Metrics:

- **Events Fired vs. Events Registered:** Total events fired this session versus events registered in the Canonical Event Table. The fired set must always be a subset of the registered set.
- **Unauthorized Event Flag:** Alerts if any event fires without a matching EVT-ID in the Event Table. This should never occur in a compliant build. Triggers a CRITICAL alert.

State Integrity Checks:

Check	Rule	Expected Status
Thorn → Redemption violation	Thorn contact must never increase redemption	Must always be CLEAN
Thorn → Win State violation	Thorn contact must never directly trigger a win state	Must always be CLEAN
Corruption out of bounds	corruption must remain in range 0-100	Must always be CLEAN
Redemption out of bounds	redemption must remain in range 0-100	Must always be CLEAN

Drift Indicator:

Status	Meaning
GREEN	No violations detected. System operating within canonical bounds.
AMBER	Warnings or edge cases detected. Review recommended.
RED	Canonical violation detected. Immediate investigation required.

Each violation is logged with: violation type, timestamp, offending state values, and suggested corrective action.

3.4 Entity Inspector Panel

The Entity Inspector provides a detailed view of individual game entities when selected by the operator. It allows drill-down inspection of any entity currently active in the game world.

Entity Type	Inspectable Properties
Petal	Type (Guilt, Desire, Rage, Mercy) · Position (x, y) · Collection status (active / collected) · Associated Event ID
Thorn Zone	Position (x, y) · Bounds (width, height or radius) · Contact status (player in zone? yes/no) · Cumulative corruption applied this session
Player	Position (x, y) · Velocity · Current contact state (in thorn zone? yes/no) · Petals collected (list by type)
Altar	Position (x, y) · Locked/unlocked status · Approach events logged (list of timestamps)

3.5 Session Summary Panel

The Session Summary is displayed at session end (game over) or on demand via operator action. It provides a comprehensive audit of the completed session.

Field	Description
Total Session Duration	Wall-clock time from session start to session end
Final State Snapshot	corruption, redemption, score, ending_type at session end
Events Fired	Total count with breakdown by category (COLLECT, HAZARD, THRESHOLD, TERMINAL, NARRATIVE)
Petals Collected	Breakdown by type: Guilt, Desire, Rage, Mercy

Field	Description
Thorn Contact Episodes	Count of distinct contact events · Total corruption applied via thorns
Compliance Status	Clean or Violations Detected — with full violation details if applicable
Narrative Log	Full ordered transcript of all narrative outputs dispatched during session
Session Grade	Informational grade based on compliance cleanliness — not gameplay skill. Canonical sessions with zero violations receive highest grade.

4. Operator Controls

The Operator Surface provides both observation and intervention capabilities. Controls are stratified into read-only actions (always available) and write actions (development/debug mode only, guarded by confirmation gates).

4.1 Read-Only Controls (Always Available)

- **Pause/Resume Observation** — Freeze the Operator Surface display without affecting gameplay. The game continues running; the surface holds its current readout for inspection.
- **Filter Event Log** — Show or hide event categories (COLLECT, HAZARD, THRESHOLD, TERMINAL, NARRATIVE) individually or in combination.
- **Select Entity** — Click or designate any game entity to load its detailed view in the Entity Inspector Panel.
- **Export Session Log** — Export the current Event Log as JSON or CSV.
- **Export State Snapshot** — Capture and export the current state of all canonical variables as a JSON object.

4.2 Write Controls (Debug/Development Mode Only — Guarded)

Warning

All write controls require explicit confirmation and permanently flag the session as non-canonical. Write controls are disabled in production and release builds.

Control	Action	Confirmation Required
Override State Variable	Set any canonical state variable (e.g., corruption, redemption) to a specified value	"You are about to override canonical state. This action is logged and may invalidate session compliance. Confirm?"
Spawn Entity	Add a petal or thorn zone at specified coordinates. Entity is logged as non-canonical.	Yes — entity tagged as operator-spawned
Trigger Event Manually	Fire a specific EVT-ID without its trigger condition being met. Logged as operator-triggered, not condition-triggered.	Yes — event tagged as operator-triggered
Reset Session	Clear all state, restart the game loop from initial conditions	Yes — previous session data is finalized before reset
Toggle God Mode	Player takes no corruption from thorn contact. For testing purposes only. Permanently flags session as non-canonical.	Yes — session permanently marked non-canonical

4.3 Control Governance

- All write actions are logged in the Event Log with an **OPERATOR** category tag, distinct from all game-generated categories.

- Sessions with any operator override are permanently flagged as "**Non-Canonical Session**" in the Session Summary. This flag cannot be removed.
- Write controls are **disabled** in production and release builds. They are available only in development and QA builds, enforced at the build configuration level.

5. Alert System

The Operator Surface employs a four-tier alert hierarchy. Alerts appear as a notification bar at the top of the Operator Surface and are logged permanently in the Event Log.

Level	Color	Trigger Conditions
INFO	Gray	Routine state changes, milestone crossings, petal collections. Standard operational activity.
WARNING	Amber	Corruption crossing 50 or 75 thresholds. Sustained thorn contact. Altar approach while altar is locked.
CRITICAL	Red	Unauthorized event detected (no matching EVT-ID). State variable out of bounds. Canonical violation of any kind.
TERMINAL	Crimson Pulse	Game over triggered — player consumed (corruption = 100) or altar reached (redemption $\geq$ 65, altar unlocked).

Note

All alerts are persisted in the Event Log regardless of whether the operator has the notification bar visible. No alert is ever discarded silently.

6. Data Export Formats

The Operator Surface supports structured data export for post-session analysis, compliance auditing, and integration with external tooling.

Export Type	Format	Contents
Session Log	JSON	Array of all events with full metadata: timestamp, Event ID, trigger condition, state effects, narrative output
State Snapshot	JSON	Object containing all canonical state variables at the moment of capture. Can be triggered at any point during a session.
Compliance Report	JSON	Violation list, drift indicator history over session duration, entity audit trail
Narrative Transcript	Plain Text	Ordered list of all narrative log entries dispatched during the session, with timestamps
Full Session Bundle	ZIP	Contains all of the above: Session Log, State Snapshot, Compliance Report, and Narrative Transcript

7. Implementation Phases

The Operator Surface is delivered across three phases, each expanding capability while maintaining backward compatibility with the governing principles defined in Section 2.

Phase 1 — v0.1 (Prototype)

Component	Implementation
Surface Form	Minimal overlay or separate browser dev-tools panel
State Inspector	Display corruption, redemption, score, game_over, ending_type as plain text readouts
Event Log	Structured console.log output, filterable in browser developer tools
Compliance	Basic assertions that throw warnings on canonical violation
Entity Inspector	Not implemented in this phase
Export	Not implemented — manual copy from console

Phase 2 — Modular

Component	Implementation
Surface Form	Dedicated Operator Surface UI panel, separate from game canvas
State Inspector	Full variable readout with color-coded threshold bands
Event Log	Scrolling log panel with category color codes and filters
Compliance	Live Canonical Compliance dashboard with drift indicator
Entity Inspector	Fully operational — select any entity for detailed inspection
Export	JSON export for session log and state

Component	Implementation
	snapshots

Phase 3 — Production Cockpit

Component	Implementation
Surface Form	Full cockpit UI with all panels simultaneously visible and configurable
Compliance	Real-time compliance monitoring with automated drift detection (integrates with Drift Detection Harness)
Replay	Session replay from state snapshots — deterministic playback for debugging
Access Control	Role-based access: Operator (full read/write), QA (read + limited write), Observer (read-only)
Export	Full export suite: Session Log, State Snapshot, Compliance Report, Narrative Transcript, Full Session Bundle (ZIP)

8. Integration Points

The Operator Surface does not exist in isolation. It connects to and draws from the broader Bloodrose canonical artifact ecosystem.

Artifact	Integration	Direction
Canonical Event Table	The Event Log validates every fired event against the Event Table. Any event with no matching EVT-ID triggers a CRITICAL alert. The Event Table is the source of truth for valid event identifiers.	Event Table → Operator Surface
Canonical Spec	The State Inspector	Spec → Operator Surface

Artifact	Integration	Direction
	displays variables defined in the spec. The Compliance Panel enforces spec-defined constraints (e.g., thorn rules, value bounds, altar unlock conditions).	
Drift Detection Harness (Artifact 5)	The Compliance Panel feeds violation data and drift indicators to the Drift Detection Harness for automated analysis. In Phase 3, the harness feeds results back to the Operator Surface for real-time display.	Bidirectional (Phase 3)
Contractor Contract (Artifact 2)	Session compliance reports generated by the Operator Surface serve as acceptance evidence during contractor deliverable review. A clean session report is a prerequisite for deliverable sign-off.	Operator Surface → Contract Review

9. Operator Surface Governance Rules

The following rules govern the Operator Surface itself — its behavior, its boundaries, and its relationship to the system it observes.

- The Operator Surface **must never alter game state** without explicit operator action and confirmation. Passive observation must have zero side effects on the running system.
- The Operator Surface **must never be visible to the player** in production builds. It is an operator-only instrument. Leaking operator readouts into the player experience is a governance violation.

- **All operator actions are logged and auditable.** There is no unlogged action on the Operator Surface. Every button press, every filter change, every export is recorded.
- The Operator Surface **consumes the same state as the player UI** — it does not maintain a separate state model. There is one source of truth. Two consumers.
- If the Operator Surface and the player UI **disagree on state**, the Operator Surface reading is considered authoritative. The Operator Surface reads raw state; the player UI may apply visual smoothing, animation delays, or presentational interpolation.
- The Operator Surface specification is a **governed document** subject to the same amendment process as all canonical Bloodrose documents. Changes require review and approval through the established governance pipeline.

10. Amendment Log

Amendment ID	Date	Description	Author	Approved By
AMD-000	April 21, 2026	Initial Operator Surface Spec created	System Architect	Pending Review

Bloodrose — Operator Surface Spec v0.1 · Canonical Governance Document ·

Classification: Internal — Operator Use Only

This document is governed under the Bloodrose canonical amendment process.

Unauthorized modification is prohibited.